

Aula 04 - Listas lineares encadeadas

3. Exercício - 03

- A função abaixo promete comportar-se como a função busca. Identifique o problema:

```
celula *busca (int x, celula *le) {  
    celula *p = le;  
    int achou = 0;  
    while (p != NULL && !achou) {  
        if (p->conteudo == x) achou = 1;  
        p = p->prox;  
    }  
    if (achou) return p;  
    else return NULL;  
}
```

Análise: Dentro do laço while, quando o valor x é encontrado (p->conteudo == x), é executado p = p->prox antes de sair do laço ou finalizar a função. Isso significa que, ao retornar p, está retornando o **próximo** elemento da lista, e não a célula onde o conteúdo x foi efetivamente localizado. Como a variável p é atualizada a cada iteração, no momento em que achou torna-se verdadeiro, o ponteiro p já avançou para a próxima posição da memória.

- Critique a seguinte variante da função busca:

```
celula *busca (int x, celula *le) {  
    celula *p = le;  
    while (p != NULL && p->conteudo != x)  
        p = p->prox;  
    if (p != NULL) return p;  
    else printf("x n o est na lista\n");  
}
```

Análise: Se o elemento x não for encontrado, a função entra no bloco else, imprime a mensagem e termina sem executar um comando return. Em C, uma função que promete retornar um ponteiro (celula *) deve, obrigatoriamente, retornar algo em todos os caminhos possíveis. Caso contrário, ela retornará um valor indefinido (lixo de memória), o que pode causar um travamento (*crash*) no programa que chamou a função. Uma função de busca, em bibliotecas técnicas, geralmente não deve imprimir mensagens no console (printf). O padrão esperado é que ela retorne apenas NULL em caso de falha.

5. Exercício - 05

- Por que a seguinte versão da função insere não funciona?

```
void insere(int x, celula *p) {  
    celula nova;  
    nova.conteudo = x;  
    nova.prox = p->prox;  
    p->prox = &nova;  
}
```

A variável celula nova; é declarada dentro da função. Em C, isso significa que ela é alocada na pilha (stack) e existe apenas enquanto a função insere está sendo executada. Assim que a função termina, o espaço de memória ocupado por nova é liberado para ser usado por outras partes do programa. O ponteiro p->prox = &nova; armazena o endereço de algo que "deixou de existir". É necessário alocar na memória a celula nova usando o comando malloc e sempre verificar se alocação funcionou.

Quando a função retorna, a lista encadeada agora aponta para um endereço de memória inválido. Se tentar acessar a lista depois: O valor de x pode ser sobrescrito por qualquer outra função que seja chamada depois; O programa pode tentar acessar uma área de memória que não pertence mais à lista, causando um *Segmentation Fault*.

- **Discussão:** é adequado alocar células uma a uma com malloc? Proponha alternativas baseadas na eficiência da alocação de pequenos blocos de memória.

O malloc unitário é adequado para aprendizado e para listas que mudam pouco de tamanho, ele apresenta gargalos significativos em aplicações reais.

Pool de Memória (Memory Pooling) / Arena Allocation: Em vez de pedir espaço para uma célula, você pede espaço para um bloco de, por exemplo, 100 ou 1000 células de uma vez.

- **Como funciona:** Você aloca um grande array de células. A sua função de inserção apenas retira a próxima célula disponível deste array pré-alocado.
- **Vantagem:** O custo de alocação cai para quase zero (apenas o incremento de um ponteiro) e as células ficam contíguas na memória, melhorando drasticamente o uso do cache.

Lista Livre (Free List): Se o seu programa remove e insere células com frequência, você pode reaproveitar as células removidas em vez de usar free.

- **Como funciona:** Quando uma célula é removida da lista principal, você a insere em uma "Lista Livre" (uma lista encadeada secundária). Na próxima inserção, você verifica se a Lista Livre tem algo; se tiver, usa aquela célula; se não, chama o malloc (ou retira do Pool).
- **Vantagem:** Evita o custo de desalocação e realocação constante.

Alocação Estática (Vetor Auxiliar): Para problemas onde o número máximo de elementos é conhecido ou limitado, pode-se usar um vetor estático de estruturas e gerenciar os índices como se fossem ponteiros.

6. Exercício – 06

- **Critique a função de remoção abaixo:**

```
void remove(celula *p, celula *le) {
    celula *lixo;
    lixo = p->prox;
    if (lixo->prox == NULL)
        p->prox = NULL;
    else
        p->prox = lixo->prox;
    free(lixo);
}
```

Falha Crítica: Falta de Verificação de Segurança: O erro mais grave é a ausência de verificação se p->prox existe.

- **O problema:** Se p for a última célula da lista, lixo = p->prox será NULL.
- **Consequência:** Na linha seguinte, ao tentar acessar lixo->prox, o programa tentará acessar o endereço 0x0 (NULL), resultando em um **Segmentation Fault** (o programa trava imediatamente).

Lógica Redundante: O bloco if-else é desnecessário e torna o código mais complexo do que precisa ser.

- **Crítica:** A linha p->prox = lixo->prox; funciona perfeitamente tanto se lixo->prox for um nó real quanto se for NULL.
- **Refatoração:** Se lixo->prox for NULL, p->prox receberá NULL naturalmente, marcando o novo fim da lista sem precisar de um if.

Parâmetro Inútil: A função recebe celula *le (provavelmente a cabeça da lista), mas **não o utiliza** em nenhum momento dentro do corpo da função.

- **Impacto:** Isso gera confusão sobre a assinatura da função e pode levar a erros de integração. Em C, parâmetros não utilizados devem ser removidos para manter a clareza.

Violação de Responsabilidade (Contrato): A função assume que o usuário sempre passará um p que possui um sucessor. Uma função robusta deve prever que a lista pode estar vazia ou que p pode ser o último elemento.

- **Suponha que queremos remover a primeira célula de uma lista não vazia. Critique o código:**

```
celula **p;  
p = &le;  
le = le->prox;  
free(*p);
```

Conflito de Tipagem e Semântica: A linha p = ≤ define p como um ponteiro para o ponteiro da lista.

- Se le é o ponteiro para o primeiro nó (celula *), então p armazena o endereço da variável le.
- Quando você executa le = le->prox; você está atualizando o valor de le.
- Como p aponta para le, o valor de *p (o conteúdo apontado por p) é o novo valor de le.

O Erro do free: Apagou a célula errada (ou a lista toda): Este é o erro mais crítico. Observe a ordem dos eventos:

- Faz le apontar para o segundo nó (le = le->prox).
- Agora, le guarda o endereço do segundo nó.
- Chama free(*p). Como *p é a mesma coisa que le, está liberando a memória do **segundo nó**, e não do primeiro.

Uso desnecessário de ponteiro de ponteiro: Para remover o primeiro elemento de uma lista simples, não há necessidade de usar celula p. Isso apenas adiciona complexidade e facilita erros de indireção como o visto acima.

- Proponha uma forma segura de remover uma célula de uma lista encadeada sem cabeça.

```
1  celula* remove_valor(celula *lst, int x) {
2      celula *ant = NULL;
3      celula *p = lst;
4
5      while (p != NULL && p->conteudo != x) {
6          ant = p;
7          p = p->prox;
8      }
9
10     if (p == NULL) return lst;
11
12     if (ant == NULL) {
13         lst = p->prox;
14     } else {
15         ant->prox = p->prox;
16     }
17
18     free(p);
19     return lst;
20 }
```